

Overview

In production ML, data and features are usually the highest leverage reliability layer. Models fail less often because of architecture choice than because of broken data assumptions.

A production model depends on three contracts:

1. **Data contract:** schema, semantics, freshness.
2. **Feature contract:** transformation logic and availability at inference time.
3. **Serving contract:** online requests map to the same feature definitions used in training.

When these contracts drift apart, teams observe silent quality degradation, unstable retraining, and hard-to-debug incidents.

Data Pipeline Architecture

A data pipeline moves raw events into validated, consumable datasets for training and inference.

Batch vs streaming

Batch pipelines

- Process data in scheduled windows (hourly/daily).
- Simpler recovery and cheaper for many tabular workloads.
- Better for use cases where minute-level freshness is not required.

Typical examples: churn scoring, risk reports, weekly demand forecasts.

Streaming pipelines

- Process event data continuously with low latency.
- Higher operational complexity (stateful processing, ordering, late events).
- Required when decisions must be made in near real time.

Typical examples: fraud detection, ad bidding, real-time personalization.

ETL and ELT patterns

- **ETL:** extract -> transform -> load into warehouse/serving table.
- **ELT:** extract -> load raw -> transform in warehouse/lakehouse.

ELT often improves auditability and reprocessing flexibility because raw data is retained.

Data validation and quality checks

Production pipelines need automated checks for:

- schema validity,
- null ratios,
- range constraints,
- cardinality and duplicate anomalies,
- freshness and timeliness.

Think of these checks as unit tests for data.

A practical quality score can be represented as:

$$Q = w_1 s_{\text{schema}} + w_2 s_{\text{freshness}} + w_3 s_{\text{completeness}} + w_4 s_{\text{validity}},$$

where each component is normalized to $[0, 1]$ and triggers alerts below threshold.

Feature Engineering & Feature Stores

Features transform raw data into predictive signals. In production, the challenge is consistency between training and serving.

Feature extraction, transformation, selection

- **Extraction:** collect relevant raw attributes and joins.
- **Transformation:** scaling, encoding, aggregations, windows.
- **Selection:** keep useful features while controlling complexity and leakage risk.

Offline vs online features

- **Offline features:** computed for training/historical analysis.
- **Online features:** computed or retrieved during real-time inference.

Offline and online definitions must be identical. Any mismatch causes training-serving skew.

Feature store concept

A feature store centralizes feature definitions, storage, and serving access patterns:

- unified feature registry,
- reusable transformations,
- offline and online materialization,
- metadata and lineage.

Open-source and managed patterns include Feast, cloud-native feature stores, and custom internal platforms.

Why feature stores matter

- Reduce duplicate feature engineering across teams.
- Improve consistency and discoverability.
- Enforce ownership and SLA expectations for key features.

Point-in-Time Correctness

Point-in-time correctness means training features must only use information available at prediction timestamp.

If the label timestamp is t_y , each feature value used for that row must come from data at time $\leq t_y$.

Formally, for feature f and entity e :

$$f(e, t_y) = g(\{x_e(t) : t \leq t_y\})$$

No future leakage is allowed.

Why this is critical

Leakage often creates deceptively strong offline metrics that collapse in production.

Common leakage sources:

- joining latest snapshot tables without time constraints,

- using post-event statuses as input features,
- aggregations with windows extending beyond prediction timestamp.

Feature store docs (e.g., Feast and Databricks) explicitly emphasize point-in-time joins to avoid leakage and training-serving inconsistency.

Tools & Patterns

Pipeline orchestration and processing

- **Airflow**: DAG scheduling for batch workflows.
- **Beam/Spark/Flink**: distributed batch or streaming transformations.
- **Dask**: scalable Python-first data workflows for medium-scale use cases.

Feature store architectures

Typical architecture:

1. Feature definitions in registry.
2. Offline materialization to warehouse/lake.
3. Online materialization to low-latency KV store.
4. Retrieval APIs for training joins and online serving.

Data contracts and schema evolution

Use explicit schema versioning and backward-compatible rollout plans:

- add new fields safely,
- deprecate old fields with migration windows,
- monitor producers and consumers for version drift.

Common Pitfalls

1. **Schema drift without alerting**
 - Upstream teams change fields; model pipelines fail or silently degrade.
2. **Offline-online feature mismatch**
 - Different code paths produce different values for the same feature.
3. **Backfills that break training consistency**
 - Recomputed historical features do not match original real-time logic.
4. **Weak freshness guarantees**
 - Serving uses stale feature snapshots during traffic spikes.
5. **No ownership model**
 - Critical features have no clear on-call/maintenance responsibility.

Business Case Studies & Exceptions

Case Study 1: Churn prediction with daily batch features

Scenario:

- Subscription business predicts churn daily using account and engagement signals.

Pipeline design:

1. Ingest daily snapshots.
2. Run batch transforms (rolling 7/30-day aggregates).

3. Validate quality and schema.
4. Publish feature table + training dataset.

Why batch works:

- Churn decisions are not second-by-second.
- Lower operational burden than streaming.

Exception:

- Campaign-triggered interventions may still need near-real-time updates for key signals.

Case Study 2: Fraud detection with streaming features

Scenario:

- Payments platform scores each transaction in milliseconds.

Pipeline design:

- Stream ingestion of transaction events.
- Real-time feature windows (velocity, geolocation consistency, device risk).
- Online feature store retrieval for inference.

Benefits:

- Better detection of short-lived attack patterns.

Risks:

- Late-arriving events can corrupt windowed features if watermarks and event-time handling are weak.

Exceptions and trade-offs

- Streaming is not automatically better. If decision latency requirement is minutes/hours, batch can be more reliable and cheaper.
- Feature stores add operational overhead; small teams may start with disciplined SQL + versioned transformations before adopting full platform.

Interview Questions & Answers

1. **Explain batch vs streaming pipelines.**
Batch processes data on schedules; streaming processes event-by-event with low latency and higher operational complexity.
2. **What is a feature store and why use it?**
A system for managing reusable, versioned, offline/online-consistent features for training and serving.
3. **What is point-in-time correctness?**
Ensuring each training feature value uses only information available at prediction time to avoid leakage.
4. **What causes training-serving skew?**
Differences in feature definitions, transformations, or data sources between training and inference.
5. **How do you detect schema drift?**
Compare incoming schema/column stats against contract baselines and alert on incompatible changes.
6. **What is the difference between ETL and ELT?**
ETL transforms before loading target store; ELT loads raw first and transforms in destination system.
7. **Why do data quality gates matter for ML?**
Bad data propagates silently into features and model outputs, causing downstream failures.

8. **How do feature freshness SLAs affect model quality?**
Stale features degrade prediction relevance, especially for fast-changing domains.
9. **When is a custom feature store overkill?**
Small teams with few models and slow update cycles can often succeed with strong SQL pipelines first.
10. **How do you version features?**
Track transformation code, source schema, timestamps, and feature set definitions in a registry/metadata store.
11. **How do late events affect streaming features?**
They can distort windowed aggregates unless event-time logic and watermarks are correctly configured.
12. **What is feature lineage?**
End-to-end trace of a feature from source tables/events through transformation logic to model usage.
13. **How do you prevent leakage in joined datasets?**
Use time-aware joins and enforce feature timestamps relative to label/prediction time.
14. **What is offline vs online feature parity?**
The same feature value should be reproducible in training and real-time inference contexts.
15. **What is a good first step to improve a fragile data pipeline?**
Introduce schema contracts + automated validation tests + failure alerts before adding more complexity.
16. **Why are feature ownership models important?**
They define responsibility for reliability, documentation, and incident response.
17. **How do you evaluate pipeline reliability?**
Track SLA adherence, freshness, data quality incidents, and downstream model impact.

Further Reading & Sources

- Google MLOps guidance: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- Feast point-in-time joins: <https://docs.feast.dev/getting-started/concepts/point-in-time-joins>
- Databricks point-in-time feature joins: <https://docs.databricks.com/aws/en/machine-learning/feature-store/time-series>
- Kubeflow Pipelines overview: <https://www.kubeflow.org/docs/components/pipelines/overview/>
- Azure MLOps v2 architecture: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/machine-learning-operations-v2>